

Smart Energy

Analyse et optimisation de la
consommation électrique

AIT ALI BENAÏSSA Hamza
DOUIH Zakaria
FAHIMI Yassir
RAJI Ahmed
TAHIRI Mohamed

Problématique

L'optimisation de la consommation énergétique est un défi majeur dans les maisons connectées. Comment concevoir un système intelligent capable de surveiller, analyser et contrôler efficacement l'usage de l'électricité tout en s'appuyant uniquement sur des simulations ?

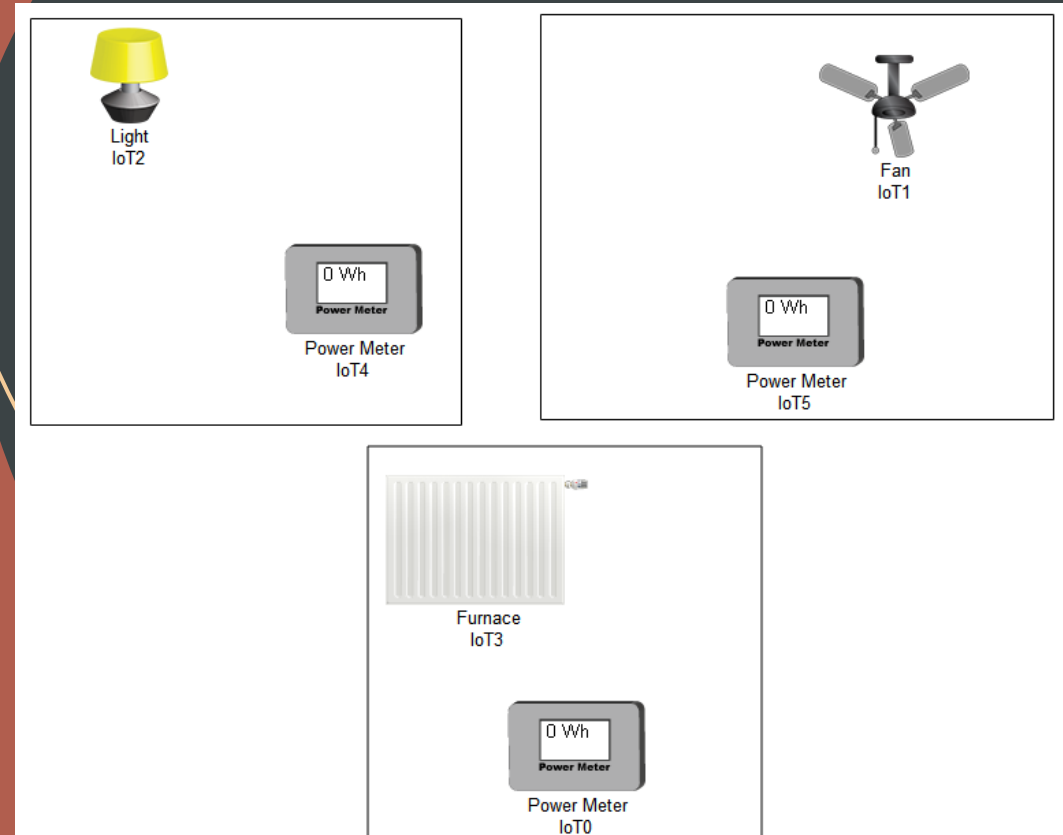
Plan

- Contexte
- Phase simulation Arduino
- Phase analyse et sauvegarde des données
- Phase Visualisation et conclusion
- Conclusion

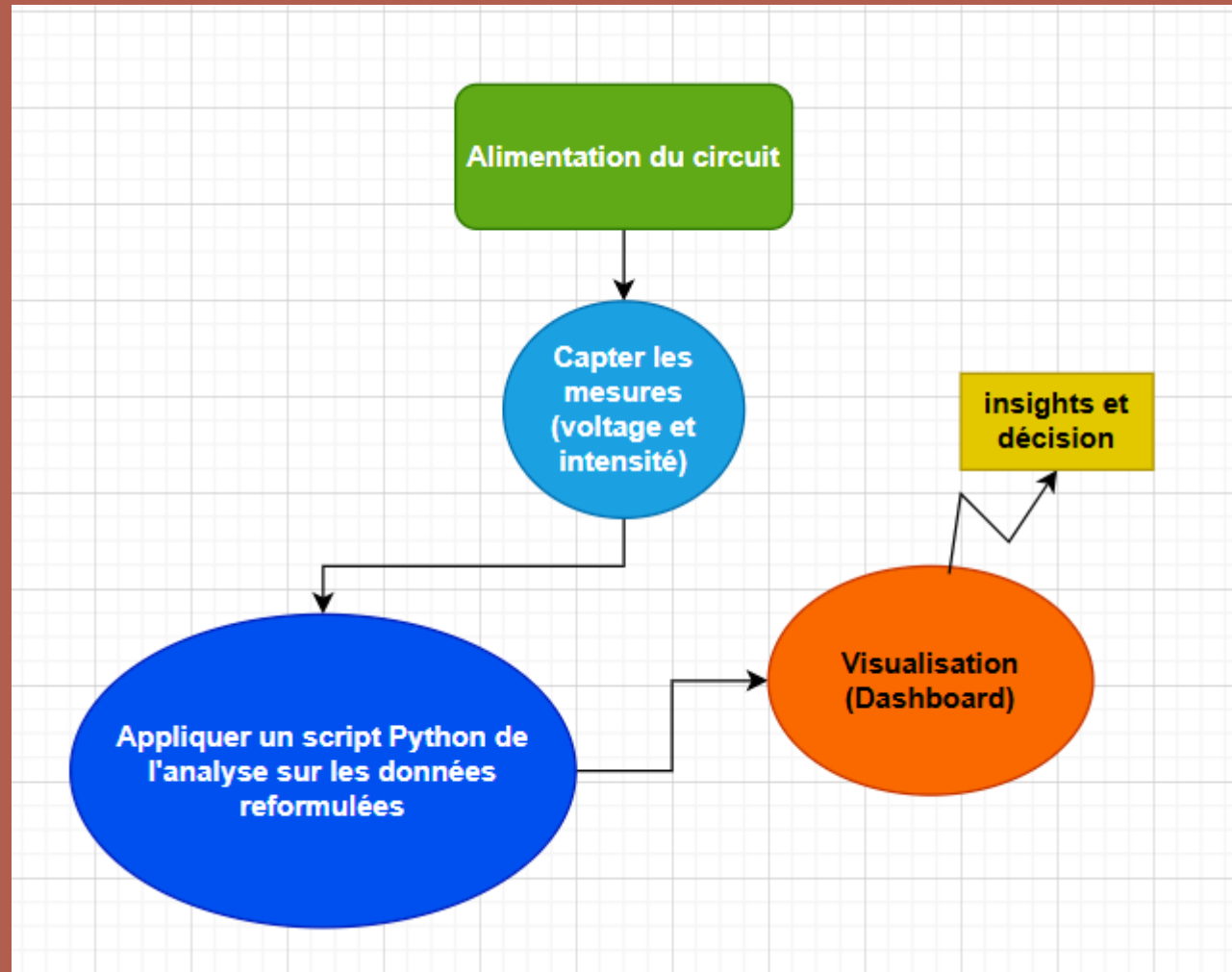
Contexte

Dans notre cas, on étudie la consommation électrique d'une maison qui comporte trois pièces. On veut répondre aux questions suivantes :

- C'est quoi la consommation totale sur une période donnée (jour, semaine, mois) ?
- Quels sont les équipements/pièces qui consomment l'énergie le plus ?
- Quelles sont les pics de consommation pendant la journée ?



Pipeline

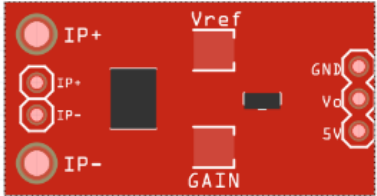
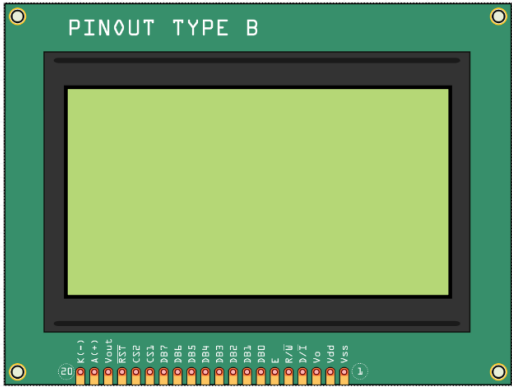
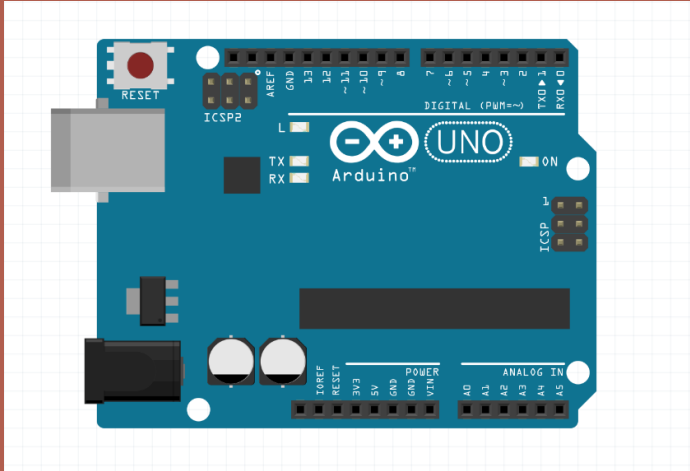


Simulation Arduino

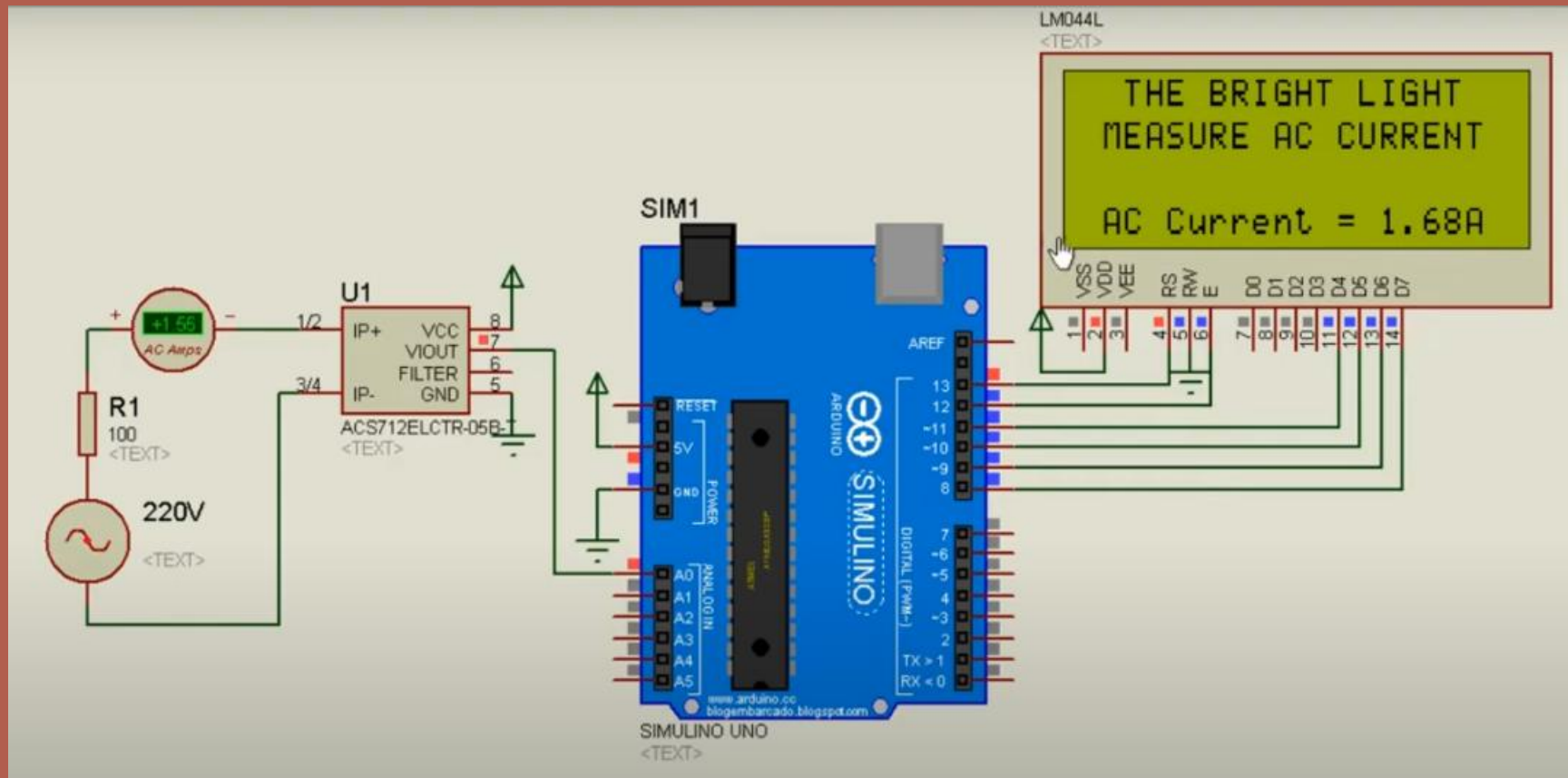
Pour simuler le circuit de chaque chambre, on travaille avec les composants suivants :

- Arduino Uno
- Capteur de courant ACS712
- Ecran LCD
- Résistance

Simulation Arduino



Simulation Arduino



Code Arduino

```
#include <LiquidCrystal.h>
LiquidCrystal lcd(13, 12, 11, 10, 9, 8);

const int Sensor_Pin = A0; int sensitivity = 185; int offsetvoltage = 2532;

void setup()
{
  Serial.begin(9600);
  lcd.begin(20, 4); lcd.setCursor(0,0); lcd.print(" THE BRIGHT LIGHT ");
  lcd.setCursor(0,1); lcd.print(" MEASURE AC CURRENT");
}
void loop()
{
  unsigned int temp=0;
  float maxpoint = 0;
  for(int i=0;i<500;i++)
  {
    if(temp = analogRead(Sensor_Pin), temp>maxpoint)
    {
      maxpoint = temp;
    }
  }
  float ADCvalue = maxpoint;
  double Voltage = (ADCvalue / 1024.0) * 5000; // Gets you mV
  double Current = ((Voltage - offsetvoltage) / sensitivity);
  double AC_Current = ( Current ) / ( sqrt(2) );

  Serial.print(" Courant = ") Serial.print(AC_Current,2); Serial.print(" A\n");
  Serial.print(" Tension = ") Serial.print(Voltage,2); Serial.print(" V\n");
  Serial.print(" Puissance = ") Serial.print(Voltage*AC_Current,2); Serial.print(" W\n");
  delay(100); //delay in milli-sec
}
```

Phase analyse et sauvegarde des données

Le code Python peut accéder aux données du console de connection Serial de l'Arduino. Ces données sont manipulées par la suite par la bibliothèque Pandas.

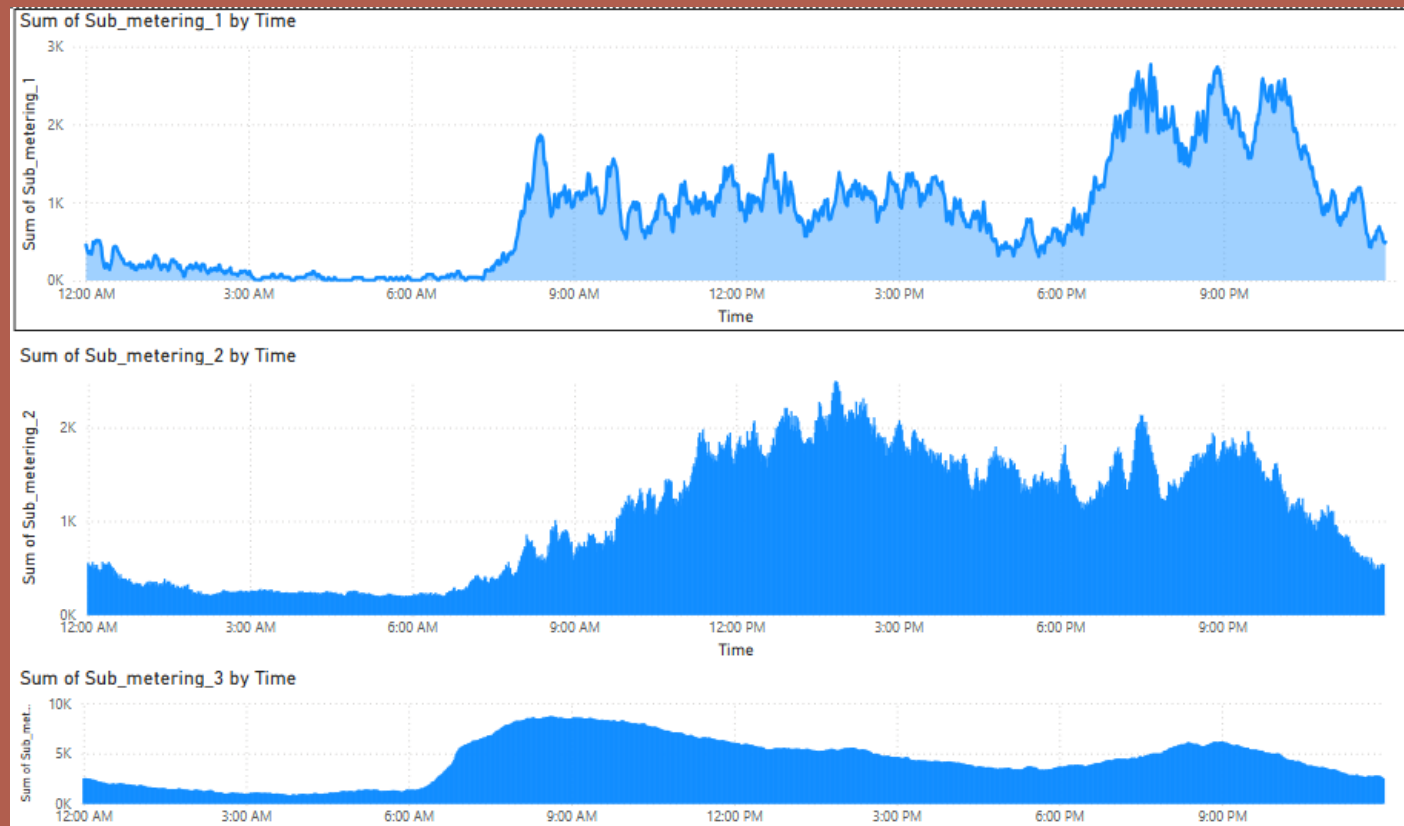
```
import serial
import pandas as pd
import numpy as np
import time
# Serial Port Configuration
SERIAL_PORT = "/dev/ttyUSB0"
BAUD_RATE = 9600
# Open Serial Connection
ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
# Create an empty DataFrame
df = pd.DataFrame(columns=["Time", "Voltage"])
try:
    while True:
        line = ser.readline().decode("utf-8").strip() # Read data from Serial
        if line:
            try:
                timestamp, voltage = line.split(",") # Parse CSV data
                voltage = float(voltage)
                timestamp = int(timestamp)
                # Append to DataFrame
                new_data = pd.DataFrame([[timestamp, voltage]], columns=df.columns)
                df = pd.concat([df, new_data], ignore_index=True)
                # Print the latest reading
                print(df.tail(1))
            except ValueError:
                print("Invalid data format:", line) # Handle errors in data
        time.sleep(1) # Wait before reading the next data
except KeyboardInterrupt:
    print("Stopping...")
    ser.close() # Close Serial Connection

# Remplacer "?" par NaN partout
df = df.replace("?", np.nan)
# Remplacer NaN par la médiane (pour les colonnes numériques)
df = df.fillna(0)
df.to_csv('res.csv', index=False)
```

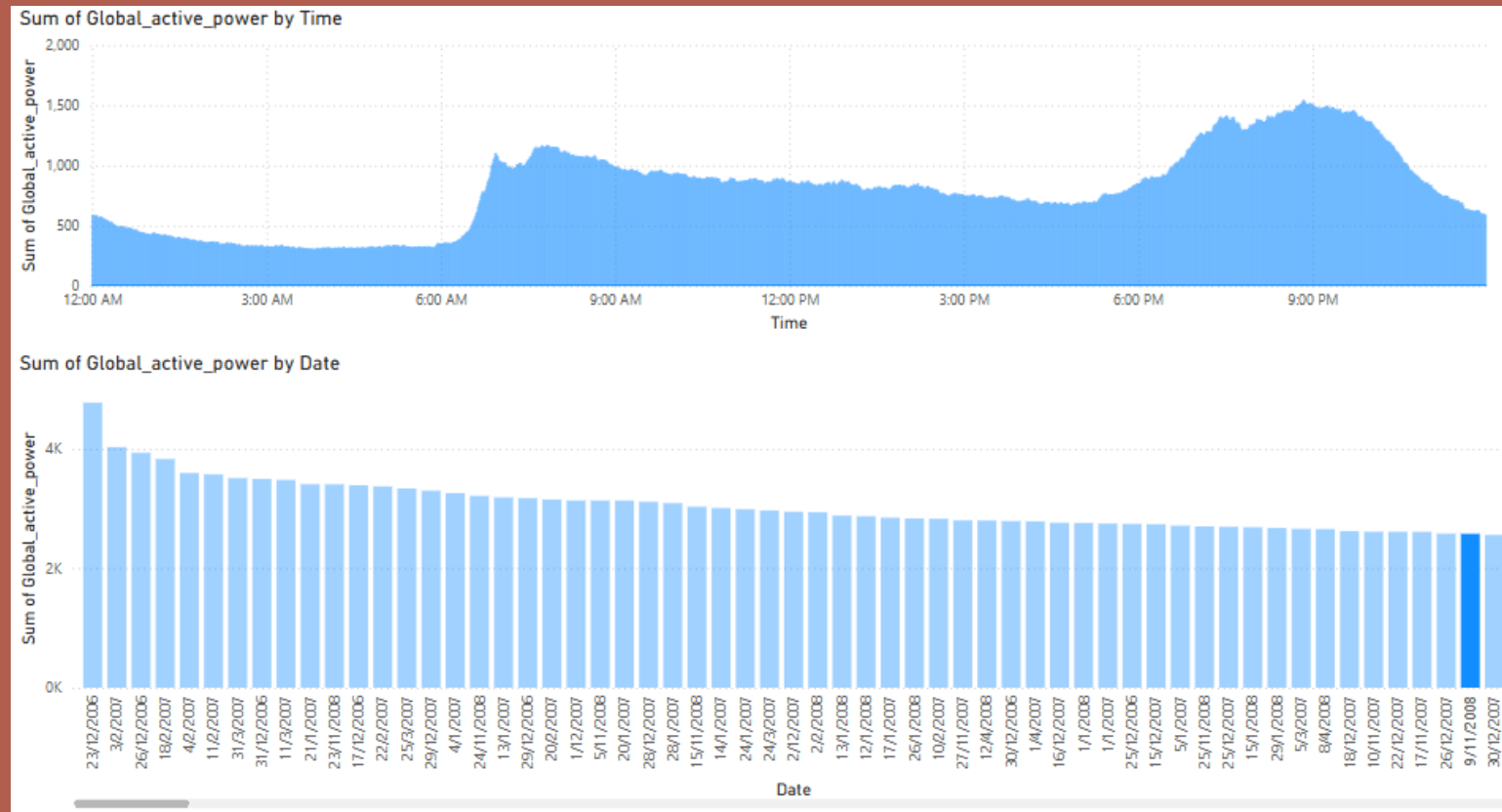
Visualisation

Le fichier CSV généré est ensuite utilisé pour visualiser les données extraites en fonction des périodes (jour, semaine, mois) en utilisant Power BI.

Visualisation



Visualisation





Merci pour votre attention !